



Sistemas Distribuidos e Internet

Tema 6

Introducción al Web Testing (Selenium)

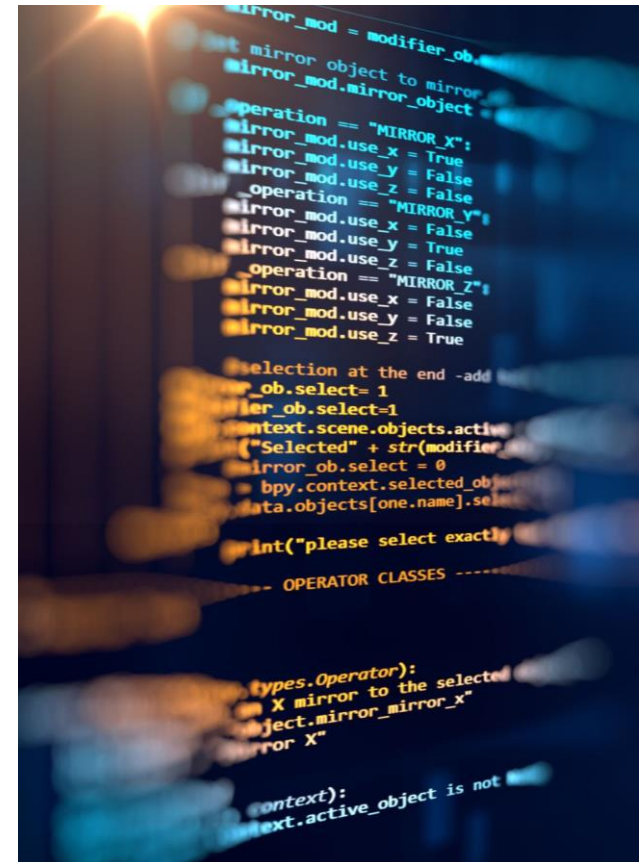


Qué es el Web testing

- Es la prueba de una aplicación web para la detección de posible fallos antes de que sea desplegada en su entorno de producción.
- Tests posibles:
 - Test funcional
 - Test de usabilidad
 - Test de interface
 - Test de compatibilidad
 - Test de rendimiento
 - Test de seguridad

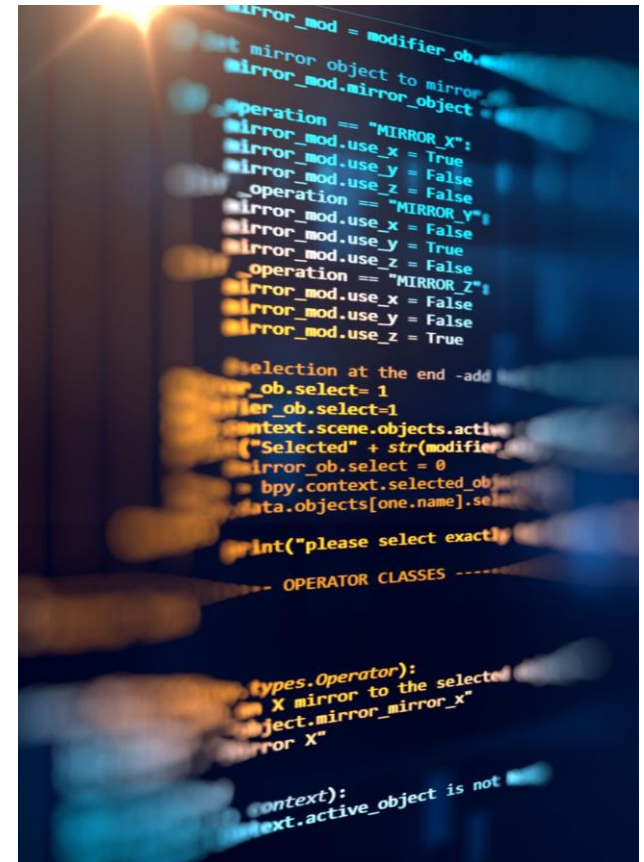
Tipos de tests – I

- **Funcional:** Prueba de todos los enlaces web, conexiones a bases de datos, envío y recepción de datos de formularios, cookies, ...
- **Usabilidad:** Es el proceso mediante el que se miden las características de la interacción computador-humano, por lo que las debilidades de esta interacción deben identificarse para corregirse.
- **Interface:** Se refiere a las conexiones entre el servidor de aplicaciones y el servidor web, y el servidor de aplicaciones y el Servidor de base de datos.
- **Compatibilidad:** Navegadores, SSOO, Disp. Móviles e Impresión.



Tipos de tests – II

- Rendimiento:
 - Pruebas de **carga**: un volumen de usuarios/conexiones, datos gestionados, conexiones a la BD, alta carga en páginas concretas
 - Pruebas de **estrés**: Exponer al sistema a valores límite de demanda de recursos y ver como responde. Los puntos críticos suelen ser campos de entrada, y areas de registro y login.
- Seguridad:
 - Uso de URLs internos directos sin identificarse.
 - Acceder a URLs para otro rol diferente al que se está identificado.
 - Ver la reacción a valores incorrectos en los formularios de login.
 - Acceso a directorios de recursos de descarga sin acceder a los enlaces de descarga.
 - Test CAPTCHA para scripts de login automático.
 - Test SSL, aviso cuando se accede a un URL https desde URLs http (no segura) y viceversa.
 - Todas las transacciones, mensaje de error y avisos de seguridad deben quedar reflejados en los archivos de log.



Qué vamos a hacer nosotros

- Elegir una herramienta que nos permita de una forma productiva realizar todos estos tipos de tests.
- ¿Qué tests vamos a diseñar?: Pues nos centraremos en
 - **Funcionales**, pero
 - También veremos de **Seguridad e Interface**.





Elección de herramientas OpenSource

Framework de pruebas unitarias

- **JUnit**, TestNG, Mocking, Matching, Spock.

Framework de pruebas funcionales

- **Selenium4**, CasperJS, Capedit, Kantoo, Phantomjs, TestCafe, ...

Navegador

- **Firefox**, Edge, Chrome, Firefox

Pruebas Unitarias:

Junit vs TestNG

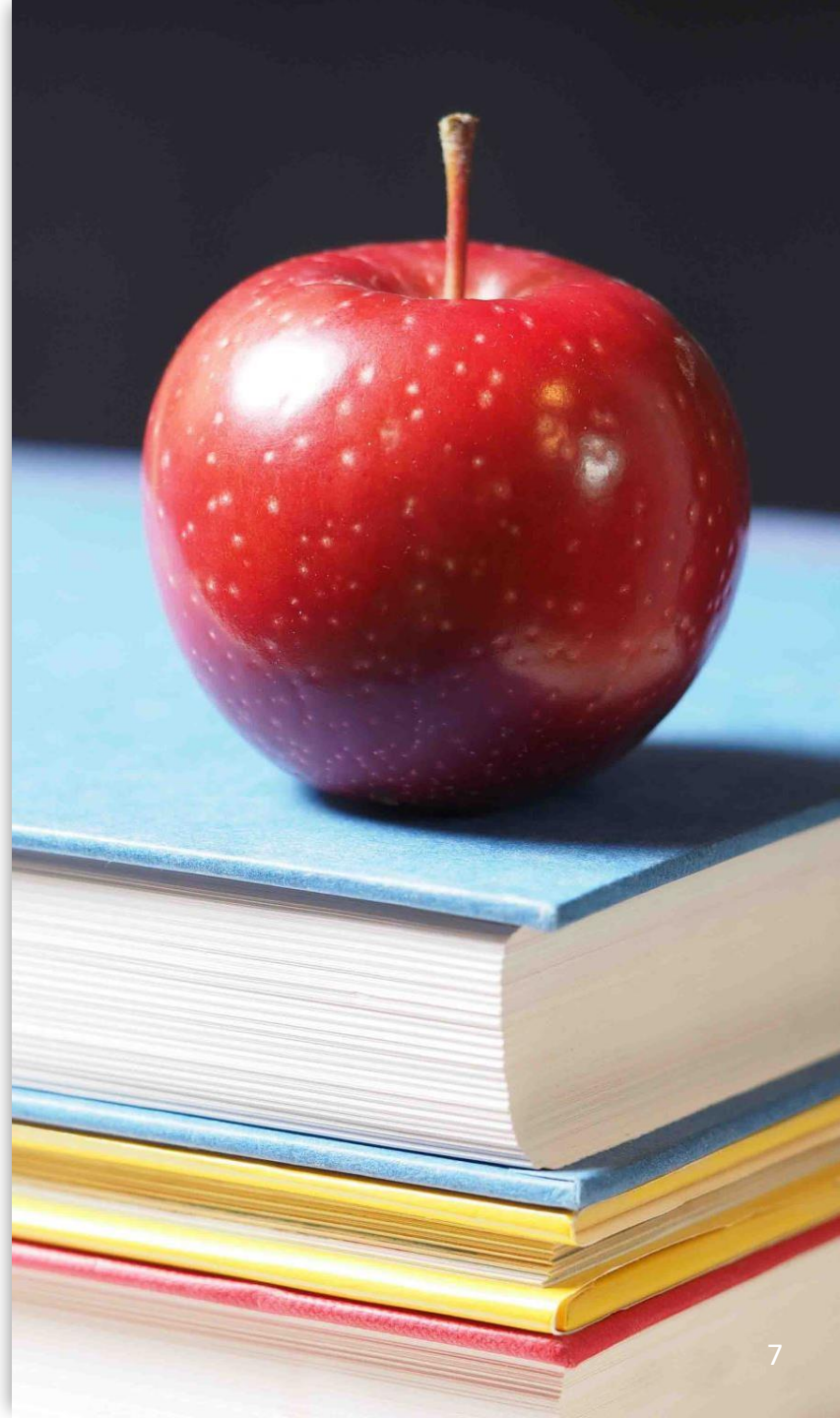
Similitudes

- OpenSource
- TestNG fue inspirado en Junit
- Son las librerías de testing más populares (ambas en el top 20)
- En **2024** Maven Central Repository Downloads
 - Junit presentó 133847 artefactos
 - TestNG 12025 artefactos

Diferencias

- JUnit: Más es más maduro y presenta una comunidad mucho mayor.
- testNG: Más potente

```
@Test(threadPoolSize = 3, invocationCount = 9)
public void testSomething() {
    ...
}
```



Pruebas funcionales

Elección de la versión de Selenium:

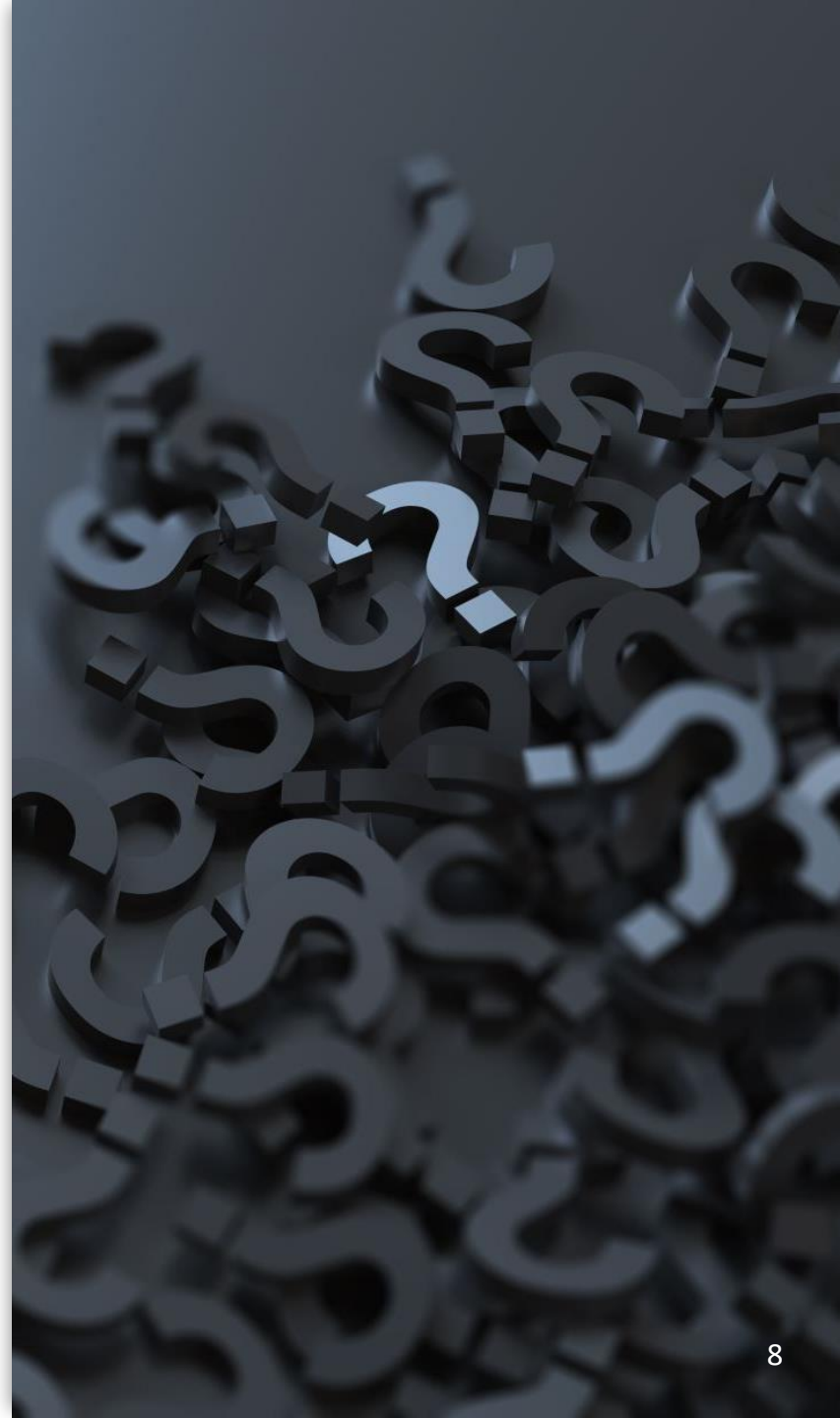
- Selenium3/4: exige instalar un driver específico según la subversión x de Selenium3.x y la versión de navegador, además de una pequeña configuración.
 - Por ejemplo para Selenium **3.141/4.28.1/Firefox135** y superiores se debe instalar geckodriver 0.30

→ Se ha seleccionado **Selenium 4.28.1**

Elección de navegador:

- Ambas versiones de Selenium disponen de soporte para los navegadores más populares: **Firefox**, Chrome, Edge y Safari.
- Elegiremos Firefox por ser la alternativa de Referencia de Selenium 4.

→ Se ha seleccionado **Firefox 135 o superior**



Componentes de Selenium

- **Selenium IDE**. Extensión de Firefox que permite grabar, editar y depurar pruebas. Permite exportar las pruebas grabadas a código **Selenese** (nativo de Selenium) o bien código Java, Ruby, Python y C# basado en Selenium API Client.
- **Selenium API Client**. API para interactuar con Selenium desde código cliente.
 - **Selenium 4 presenta una nueva API basada en WebDriver.**
- **Selenium WebDriver**. Componente de Selenium 4 API Cliente. Controlador del navegador que permite enviar comandos al propio navegador para realice acciones como si de un usuario real se tratase.
- **Selenium Grid**. Servidor que permite usar instancias de navegador ejecutándose en máquinas remotas.



Selenium Libraries



API Selenium

- **WebDriver**: Es la clase de la API de Selenium que encapsula la interacción con el navegador.

```
WebDriver driver = FirefoxDriver();  
driver.get("http://www.google.com");
```
- **Dispone de drivers** para múltiples navegadores:
Emulación de un navegador HTML
 - `WebDriver driver = new HtmlUnitDriver();`
 - Conexión real al navegador Firefox**
 - `WebDriver driver = new FirefoxDriver();`

API Selenium - II

- Localización de un elemento de interfaz mediante **WebDriver** usando alguno de los métodos de búsqueda de WebDriver: `findElement` or `findElements`:
 - El método puede retornar un **WebElement** o **List<WebElement>**.
 - En caso de no encontrar generar una **Excepción**.



API Selenium – III (Localizadores)



By.id

```
<div id="coolestWidgetEvah">...</div>  
WebElement element =  
driver.findElement(By.id("coolestWidgetEvah"));
```



By.className

```
<div  
class="cheese"><span>Cheddar</span></div>  
<div  
class="cheese"><span>Gouda</span></div>  
List<WebElement> cheeses =  
driver.findElements(By.className("cheese"));
```



By.tagName, By.name, By.LinkText,
By.partialLinkText, By.cssSelector, By.xpath

API Selenium - IV

- Como Obtener el valor de un campo input

```
WebElement dni = driver.findElement(By.id("dni"));  
dni.getText();
```

- Como rellenar un campo input

```
WebElement dni = driver.findElement(By.id("dni"));  
dni.click();  
dni.clear();  
dni.sendKeys(dnip);
```

- Como clicar un botón Submit o un enlace

```
driver.findElement(By.id("submit")).click();
```

API Selenium – V (Navegación)

- El método navigate:
 - Ir a una página:
`driver.navigate().to("http://www.example.com");`
 - Ir adelante y atrás:
`driver.navigate().forward();`
`driver.navigate().back();`
- Cookies:
 - Create a cookie
`Cookie cookie = new Cookie("key", "value");`
`driver.manage().addCookie(cookie);`
 - Manage cookies
`// By Cookie`
`driver.manage().deleteCookieNamed("CookieName");`
`// By Cookie`
`driver.manage().deleteCookie(cookie);`
`// Or all of them`
`driver.manage().deleteAllCookies();`

Esperas explícitas e implícitas

- Espera explícita:
 - Consiste en una espera por una condición antes de continuar la ejecución.
 - El peor de los casos es la **espera incondicional Thread.sleep()**
→ **PROHIBIDA EN ESTA ASIGNATURA.**
 - Con Selenium se combinan:
 - WebDriverWait con
 - ExpectedConditions

```
WebDriver driver = new FirefoxDriver();  
driver.get("http://somedomain/url_that_delays_loading");  
WebElement myDynamicElement =  
(new WebDriverWait(driver, 10))  
.until(ExpectedConditions.presenceOfElementLocated(By.id("myDynamicElement")));
```

Esperas explícitas e implícitas

- Espera implícita:
 - Le dice a `WebDriver` que sondee el árbol DOM cada cierto tiempo para encontrar un elemento sino está disponible.

```
WebDriver driver = new FirefoxDriver();  
driver.manage().timeouts().implicitlyWait(10,  
TimeUnit.SECONDS);  
driver.get("http://somedomain/url_that_delays_loading");  
WebElement myDynamicElement =  
driver.findElement(By.id("myDynamicElement"));
```

Selenium para Spring Boot

- Dos opciones:
 - Dos proyectos separados:
 - El proyecto a probar: un proyecto Spring Boot
 - El proyecto con las pruebas: proyecto Java con Selenium y Junit
 - Se incluyen las dependencias Maven para
 - » Junit 5.
 - » selenium-server3.141.59.jar
 - **Un proyecto único:** (Se ejecuta como dos proyectos diferentes).
 - Con el código Spring Boot donde habitualmente (src/main/java).
 - Con las pruebas en (src/main/test)
 - Se incluyen las dependencias Maven para
 - » Junit 5.
 - » selenium-server4.28.1

Pasos para el diseño de pruebas con Selenium

1. Ser sistemático etiquetando los atributos de las vistas, Ids, style, ... (analizar el proyecto Notaneitor final).
2. Crear un PageObject por vista o conjuntos de vistas con misma interacción.
3. Diseñar y definir los casos de test:
 - Casos válidos
 - Casos inválidos

Esquema de una suite de Pruebas

@SpringBootTest

//Ordenamos las pruebas por la anotación @Order de cada método

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)

class NotaneitorApplicationTests {

//Para MacOSX

static String PathFirefox = "/Applications/Firefox.app/Contents/MacOS/firefox";

static String Geckodriver = "/Users/delacal/selenium/geckodriver-v0.30.0-macos";

//Para Windows

//static String Geckodriver = "C:\\Path\\geckodriver-v0.30.0-win64.exe";

//static String Geckodriver = "C:\\Dev\\tools\\selenium\\geckodriver-v0.30.0-win64.exe"; **//Común a Windows y a MacOSX**

static final String URL = "http://localhost:8090";

static WebDriver driver = getDriver(PathFirefox, Geckodriver);

public static WebDriver getDriver(String PathFirefox, String Geckodriver) {

System.setProperty("webdriver.firefox.bin", PathFirefox);

System.setProperty("webdriver.gecko.driver", Geckodriver);

driver = new FirefoxDriver();

return driver;

}

//Antes de la primera prueba

@BeforeAll

static public void begin() {

}

//Al finalizar la última prueba

@AfterAll

static public void end() {

//Cerramos el navegador al finalizar las pruebas

driver.quit();

}

//Antes de cada prueba se navega al URL home de la aplicación

@BeforeEach

public void setUp() {

driver.navigate().to(URL);

}

//Después de cada prueba se borran las cookies del navegador

@AfterEach

public void tearDown() {

driver.manage().deleteAllCookies();

} }

Esquema de un caso de prueba

@Test

@Order(1)

public void metodo_prueba

{

//Paso1. Solicitud de página

Driver.get(URL)

//Paso2 .Esperamos carga de pagina.

elementos = SeleniumUtils. *waitLoadElementsBy(driver, tipo_elemento, cadena, timeout);*

//Paso3. Interacción con la pagina ... Pinchar, rellenar, ...

elementos.get(0).click(); // Por ejemplo

//Paso4. Esperar por la respuesta a la interacción

elementos = SeleniumUtils. *waitLoadElementsBy(driver, tipo_elemento, cadena, timeout);*

//Paso5. Assert de comprobacion.

Asserttrue("No se obtuvo el resultado esperado", condicion_basada_elementos);

//Empezar en Paso 3. de Nuevo si es necesario según la prueba.

}

SeleniumUtils

Ver el código del proyecto a probar

textsPresentOnPage

textsNotPresentOnPage

waitTextsNotPresentOnPage

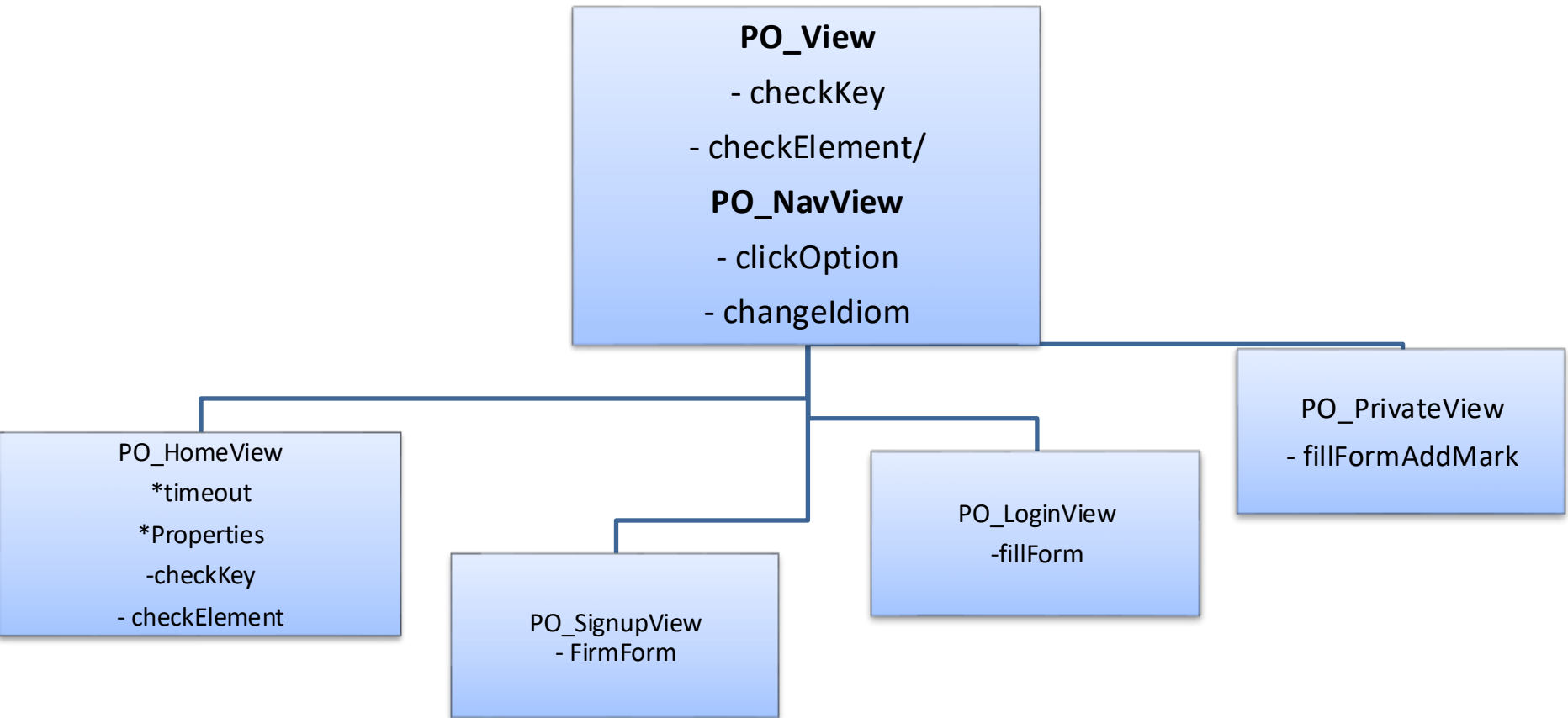
waitLoadElementsByXpath

waitLoadElementsBy

waitSeconds



Propuesta de jerarquía de PO



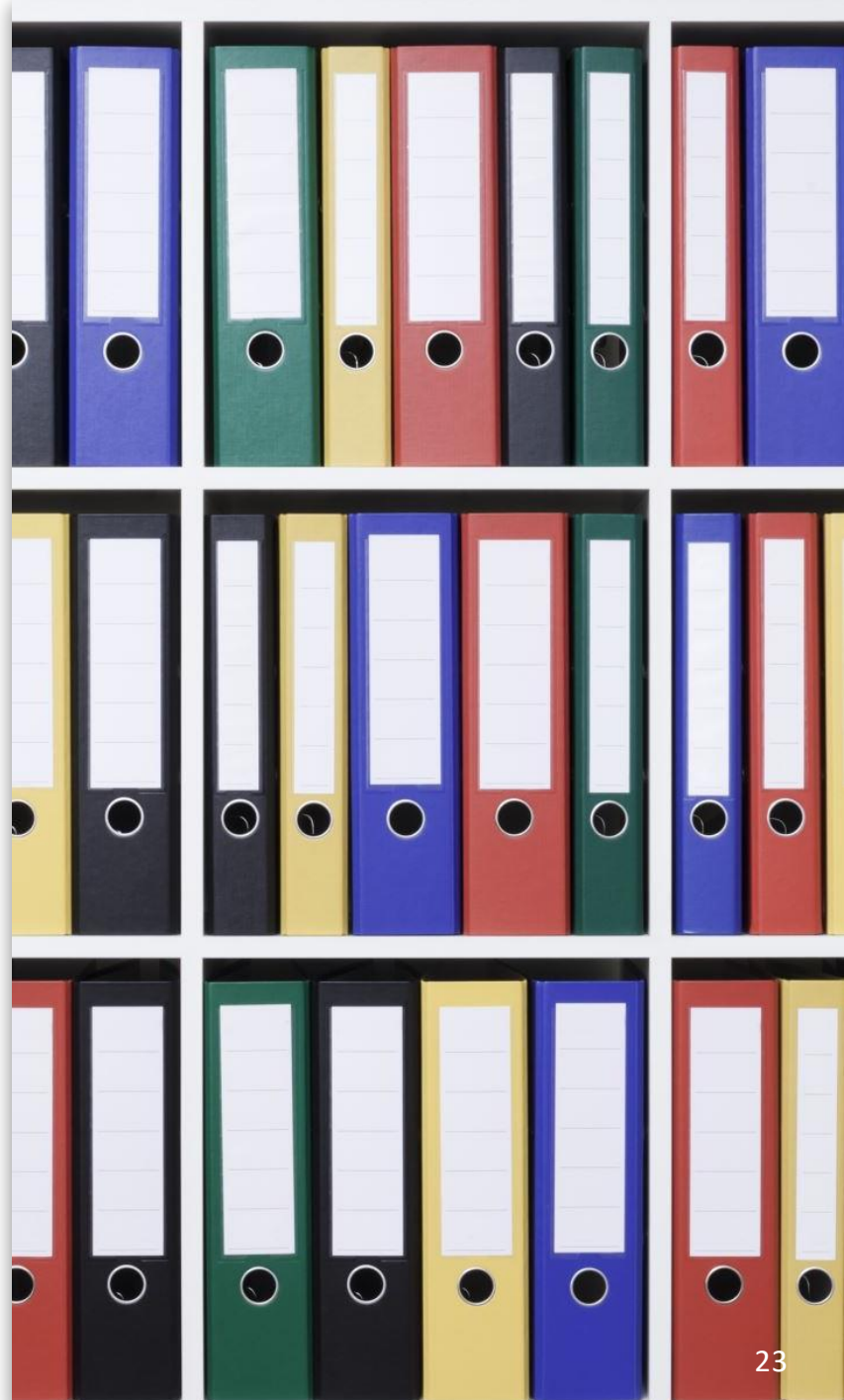
Ver el código de la solución del
proyecto a probar

Criterios de Diseño de casos de prueba

- Cada caso de prueba debe abordar sólo una entidad de datos (p.e. si se prueba el registro de un usuario, se deberá registrar un usuario por caso de prueba)
- Todos los casos deben finalizar con un aserto.

....

```
//Vamos al formulario de registro
PO_HomeView.clickOption(driver, "signup", "class",
"btn btn-primary");
//Rellenamos el formulario.
PO_SignUpView.fillForm(driver, "77777778A",
"Josefo", "Perez", "77777", "77777");
//Comprobamos que entramos en la sección privada y
nos muestra el texto a buscar
String checkText = "Notas del usuario";
List<WebElement> result =
PO_View.checkElementBy(driver, "text", checkText);
//CODIGO INCORRECTO. Falta un aserto aquí
}
```



Reseteo de base de BD tras cada caso de test

- Usando **@DirtiesContext** para Restablecer la Base de Datos
 - Esta anotación indica a **Spring** que recargue todo el contexto de la **aplicación** después de la prueba
 - Esto garantiza un **estado limpio de la base de datos** antes de cada prueba.

```
@SpringBootTest
@DirtiesContext(classMode = DirtiesContext.ClassMode.AFTER_EACH_TEST_METHOD)
public class UserServiceTest {
    @Autowired
    private UserRepository userRepository;
    @Test
    public void testCrearUsuario() {
        User user = new User("Juan Pérez", "juan@example.com");
        userRepository.save(user);

        // Verificar que el usuario se haya guardado
        assertNotNull(userRepository.findByEmail("juan@example.com"));
    }
    @Test
    public void testBaseDatosRestablecida() {
        // Si @DirtiesContext funciona, la base de datos debe estar vacía al inicio de cada prueba
        assertEquals(0, userRepository.count());
    }
}
```

Pros/Cons the DirtyContext

- Ventajas

- **Restablece automáticamente la base de datos** sin necesidad de ejecutar scripts manuales.
- Garantiza que las pruebas se ejecuten de forma aislada.
- Funciona bien con **Spring Boot** y bases de datos en memoria (**H2, PostgreSQL, etc.**).

- Desventajas

- **Lento**: Recargar el contexto de Spring puede aumentar el tiempo de ejecución de las pruebas.
- No es ideal para suites de pruebas grandes.

Alternativa con Transactional/Rollback

- Usando Cada prueba se ejecuta dentro de una **transacción** y, al finalizar, la transacción se revierte automáticamente.
- **No** se aplican los cambios en la base de datos, lo que asegura que el estado inicial se mantenga.

```
@SpringBootTest
@Transactional
public class UserServiceTest {
    @Autowired
    private UserRepository userRepository;

    @Test
    public void testCrearUsuario() {
        User user = new User("Juan Pérez", "juan@example.com");
        userRepository.save(user);

        assertNotNull(userRepository.findByEmail("juan@example.com"));
    }
}
```

Prons and Cons de Transactional

- **Pros:**

- **Rápido:** No hay necesidad de recargar el contexto de Spring, lo que hace que las pruebas sean más rápidas.
- **Simplicidad:** El rollback es automático, sin necesidad de escribir código adicional para limpiar la base de datos.
- **Pruebas aisladas:** Cada prueba se ejecuta de manera independiente y no afecta el estado de la base de datos.
- **Ideal para pruebas unitarias:** Es adecuado para pruebas que verifican la lógica de negocio a nivel de transacciones.

- **Cons:**

- **No válido para pruebas de integración complejas:** No es adecuado para pruebas que requieren persistencia real en la base de datos.
- **Limitado a bases de datos transaccionales:** Solo funciona con bases de datos que soportan transacciones (e.g., PostgreSQL, MySQL).
- **No afecta cachés:** Los datos almacenados en cachés fuera de la transacción no se restablecen.

Recomendaciones Prácticas

- **Usa @Transactional** cuando:
 - Tus pruebas se centran en la lógica de negocio.
 - Necesitas mantener la base de datos limpia y tus pruebas no dependen de caché externo.
- **Usa @DirtyContext** cuando:
 - Necesitas un contexto completamente nuevo para evitar efectos secundarios.
 - Estás realizando pruebas de integración que cubren múltiples capas del sistema.
- Se pueden combinar según las necesidades de cada test

Sistemas Distribuidos e Internet

Tema 4

Introducción al Web Testing (Selenium)